

Mappa Architettuale

Il sistema è strutturato secondo tre livelli cooperanti:

1. **Control Plane (Raft)**: garantisce la consistenza, l'elezione sicura del Leader, la replica del log a maggioranza e lo stato deterministico dei job.
2. **Data Plane (Execution & Scheduling)**: riceve i calcoli, li distribuisce secondo la politica di min-load, esegue il codice Java nei thread pool e gestisce i crash dei worker.
3. **Network & Discovery**: permette la scoperta automatica dei nodi e la tolleranza ai guasti dei client.

1. Modello Dati, DTO e Contratti di Rete

[`SerializableSupplier.java`]

(file:///home/chiera/DS_Project_JAVA/DS_Project/src/main/java/SerializableSupplier.java)

- **Ruolo**: Interfaccia funzionale fondamentale per consentire la **Code Mobility** (mobilità del codice) tramite Java RMI.
- **Descrizione del Codice**:
Estende contemporaneamente `java.util.function.Supplier<T>` e `java.io.Serializable`. Essendo serializzabile, permette al Client di definire una lambda expression arbitraria (ad esempio un calcolo matematico) e di inviarla come flusso di byte attraverso la rete all'Executor remoto per l'esecuzione.

[`Job.java`](file:///home/chiera/DS_Project_JAVA/DS_Project/src/main/java/Job.java)

- **Ruolo**: Rappresenta l'unità di calcolo sottomessa dal Client e gestita dal cluster.
- **Strutture Dati e Campi**:
 - `jobId` (String): ID univoco assegnato al job.
 - `clientId` (String): identificativo univoco del client sottomettitore (es. `client-9c509a05`).
 - `requestId` (long): numero sequenziale della richiesta del client, usato per la deduplicazione.
 - `attempt` (int): contatore dei tentativi di esecuzione (inizializzato a 1; incrementato in caso di riassegnazione post-crash).
 - `task` (`SerializableSupplier<T>`): il codice Java da eseguire.
- **Metodi Chiave**:
 - `execute()`: invoca `task.get()` ed esegue il lavoro, restituendo il risultato generico `T`.
 - Costruttori multipli: garantiscono piena retrocompatibilità sia per chiamate legacy sia per invocazioni arricchite con metadati di tracciamento.

[`LogEntryJob.java`]

(file:///home/chiera/DS_Project_JAVA/DS_Project/src/main/java/LogEntryJob.java)

- **Ruolo**: È il record formale memorizzato all'interno del Raft Replicated Log. Incapsula una transizione di stato del cluster.
- **Strutture Dati e Campi**:
 - `term` (int): termine Raft in cui il Leader ha registrato il comando.
 - `index` (int): indice sequenziale (1-based) nel log.
 - `type` (`LogEntryJob.Type`): tipo di evento:
 - `CREATE_JOB`: registrazione del job e dei dati client.
 - `ASSIGN_JOB`: decisione dello scheduler di assegnare il job a un nodo.
 - `START_JOB`: inizio computazione nel thread pool del worker.
 - `COMPLETE_JOB`: completamento del calcolo con esito positivo.

- `FAIL_JOB`: fallimento computazionale con errore.
- `REQUEUE_JOB`: rimessa in coda del job a seguito del crash del worker assegnatario.
- `jobId` (String): identificativo del job associato.
- `payload` (Object): metadati specifici per quel tipo di entry (l'oggetto `Job`, il nodo assegnatario, il risultato del calcolo, ecc.).

[`JobMetadata.java`]

(file:///home/chiara/DS_Project_JAVA/DS_Project/src/main/java/JobMetadata.java)

- **Ruolo:** Rappresenta la vista applicata di un job all'interno della [`JobStateMachine`] (file:///home/chiara/DS_Project_JAVA/DS_Project/src/main/java/JobStateMachine.java).
- **Strutture Dati e Campi:**
 - `state` (`State`): stato corrente (`SUBMITTED` , `ASSIGNED` , `RUNNING` , `COMPLETED` , `FAILED`).
 - `assignedNodeId` (String): nodo attualmente incaricato dell'esecuzione (o `null` se in attesa).
 - `attempt` (int): tentativo corrente (usato per invalidare risposte ritardate di worker riavviati).
 - `result` (Object): risultato finale del calcolo al termine del job.
 - `errorMessage` (String): eventuale messaggio di eccezione in caso di fallimento.
 - `createdAt` , `lastUpdated` (long): timestamp per metriche di latenza.
- **Funzionamento:**
Tutti i metodi di mutazione (`setState` , `setAssignedNodeId` , `setResult` , ecc.) sono `synchronized` per garantire la visibilità thread-safe concorrente tra lo Scheduler e le interrogazioni del Client.

[`NodeInfo.java`]

(file:///home/chiara/DS_Project_JAVA/DS_Project/src/main/java/NodeInfo.java)

- **Ruolo:** Descrittore di rete e stato di un nodo del cluster.
- **Campi:**
 - `nodeId` , `ipAddress` , `port` : coordinate fisiche per raggiungere il registro RMI.
 - `activeJobs` : stima del carico locale.
 - `term` : termine Raft in cui il nodo opera.
 - `isLeader` : booleano che indica se il nodo ritiene di essere Leader.
- **Funzionamento:**
Implementa `equals` e `hashCode` basandosi unicamente su `nodeId` , consentendo l'uso efficiente all'interno di `Set` e `Map` per deduplicare le registrazioni del cluster.

[`VoteResponse.java`]

(file:///home/chiara/DS_Project_JAVA/DS_Project/src/main/java/VoteResponse.java)

- **Ruolo:** Pacchetto di risposta restituito dalla chiamata remota Raft `requestVote` .
- **Campi:**
 - `term` (int): il termine corrente del votante (permette al candidato di retrocedere a Follower se il votante ha un termine più alto).
 - `voteGranted` (boolean): `true` se il voto è stato concesso, `false` altrimenti.

[`AppendEntriesResponse.java`]

(file:///home/chiara/DS_Project_JAVA/DS_Project/src/main/java/AppendEntriesResponse.java)

- **Ruolo:** Pacchetto di risposta restituito dalla chiamata remota Raft `appendEntries` .
- **Campi:**
 - `term` (int): termine corrente del ricevente.

- `success` (boolean): `true` se la verifica di consistenza su `prevLogIndex` e `prevLogTerm` ha avuto successo e le entry sono state salvate.
- `matchIndex` (int): l'indice massimo del log allineato con successo (usato dal Leader per calcolare il Quorum).

[`RemoteExecutorInterface.java`]

(file:///home/chiera/DS_Project_JAVA/DS_Project/src/main/java/RemoteExecutorInterface.java)

- **Ruolo:** Contratto RMI pubblico del sistema che estende `java.rmi.Remote`.
- **Dichiarazioni dei Metodi:**
 - `VoteResponse requestVote(int term, String candidateId, int lastLogIndex, int lastLogTerm)`: RPC di elezione Raft.
 - `AppendEntriesResponse appendEntries(...)`: RPC di replica e heartbeat del Leader.
 - `String submitJob(Job<?> job)`: punto d'ingresso per la sottomissione da parte dei client.
 - `void executeJob(Job<?> job, int attempt)`: comando inviato dal Leader al Worker per avviare il calcolo.
 - `void notifyJobCompletion(...)`: callback inviata dal Worker al Leader al termine del calcolo.
 - `Object getJobResult(String jobId)`: consultazione del risultato (usata dal Client).

[`ClusterStatus.java`]

(file:///home/chiera/DS_Project_JAVA/DS_Project/src/main/java/ClusterStatus.java)

- **Ruolo:** Interfaccia di sola lettura per ispezionare l'elenco dei nodi vivi (`getAliveNodes()`) e verificare se un nodo è attivo (`isAlive(nodeId)`).

2. Il Control Plane: Consenso Raft e Persistenza

[`RaftLog.java`](file:///home/chiera/DS_Project_JAVA/DS_Project/src/main/java/RaftLog.java)

- **Ruolo:** Gestore del log replicato e della sua persistenza duratura su disco (stable storage).
- **Architettura Interna:**
 - Mantiene una lista in memoria `List<LogEntryJob> entries` indicizzata a partire da `1`.
 - L'indice `0` contiene una entry **sentinella fittizia** (`term=0`) per consentire la verifica corretta di `prevLogIndex=0` e `prevLogTerm=0` quando il log è vuoto.
 - Persiste i dati su `<nodeId>_raft_log.dat`.
- **Metodi Principali:**
 - `append(term, type, jobId, payload)`: crea una nuova entry, la assegna all'indice progressivo, la persiste su disco in modo atomico (tramite file `.tmp` e rename) e ne restituisce l'indice.
 - `truncate(fromIndex)`: rimuove dal log tutte le entry a partire da `fromIndex` fino alla fine. È l'operazione cardine con cui Raft risolve le divergenze tra leader e follower.
 - `isUpToDate(candidateLastLogIndex, candidateLastLogTerm)`: implementa la regola di **Election Safety** (Sezione 5.4.1 del paper Raft). Valuta se il log del candidato è almeno aggiornato quanto quello locale.
 - `loadFromDisk()`: ricarica la lista dal file binario all'avvio del nodo (Crash Recovery).

[`JobStateMachine.java`]

(file:///home/chiera/DS_Project_JAVA/DS_Project/src/main/java/JobStateMachine.java)

- **Ruolo:** La Replicated State Machine applicativa. Deriva in modo deterministico lo stato del cluster a partire dalle entry committate nel log.
- **Strutture Dati:**
 - `jobs` (`Map<String, JobMetadata>`): mappa in memoria di tutti i job noti nel cluster.
 - `clientRequests` (`Map<String, Map<Long, String>>`): tabella di deduplicazione `clientId -> (requestId -> jobId)`.
 - `lastAppliedIndex`: indice della entry più alta applicata.

- **Metodi Principali:**

- `apply(LogEntryJob entry)` : esegue la transizione di stato in base a `entry.getType()` . Viene invocato **solo su entry che hanno raggiunto il Quorum**.
- `getExistingJobId(clientId, requestId)` : controlla se una richiesta client è già stata registrata in precedenza, realizzando l'idempotenza.
- `getJob(jobId)` : fornisce i metadati del job per lo Scheduler e per le risposte ai client.

[**ClusterManager.java**]

(file:///home/chiara/DS_Project_JAVA/DS_Project/src/main/java/ClusterManager.java)

- **Ruolo:** È il cervello distribuito del nodo. Implementa il protocollo di consenso Raft, il failure detector e il coordinamento delle comunicazioni.
- **Stato Raft Mantenuto:**
 - `state` : ruolo attuale (`FOLLOWER` , `CANDIDATE` , `LEADER`).
 - `currentTerm` , `votedFor` : persistiti su `<nodeId>_raft.properties` .
 - `raftLog` : riferimento al log persistente.
 - `jobStateMachine` : riferimento alla macchina a stati applicativa.
 - `nextIndex` , `matchIndex` : registri volatili del Leader per tracciare la sincronizzazione di ogni follower.
- **Algoritmi e Meccanismi Implementati:**
 - **Failure Detection & Election Timeout (`checkFailures`):** un timer periodico (1 sec) scarta i nodi che non inviano heartbeat da >15s. Se scade l'election timeout casuale (5-10 sec), scatena `startElection()` .
 - **Elezione Sicura (`startElection` e `handleRequestVote`):**
Il candidato incrementa il termine, vota per sé e interpella i pari inviando `lastLogIndex` e `lastLogTerm` .
Il votante concede il voto solo se il termine è valido e `raftLog.isUpToDate(...)` restituisce `true` .
 - **Replica del Log (`replicateToFollower`):**
Se il nodo è Leader, ogni 1.5 secondi (tramite `replicationScheduler`) invia `appendEntries` a ciascun follower con le entry a partire da `nextIndex[peerId]` .
 - **Verifica di Consistenza del Follower (`handleAppendEntries`):**
Il follower verifica `prevLogIndex` e `prevLogTerm` . Se c'è discrepanza, risponde `success=false` . In caso di successo, tronca eventuali conflitti, appende le nuove entry e resetta il proprio election timer (soppressione dell'elezione).
 - **Avanzamento del Quorum (`checkQuorumCommit`):**
Il Leader calcola se un indice $N > commitIndex$ è stato replicato su almeno $\lfloor expectedClusterSize/2 \rfloor + 1$ nodi. Se sì, avanza `commitIndex` , invoca `applyCommittedEntries()` e risveglia i thread in attesa con `notifyAll()` .
 - **Apply Loop (`applyCommittedEntries`):**
Scorre in sequenza tutte le entry committate tra `lastApplied + 1` e `commitIndex` e le applica alla `JobStateMachine` .

3. Il Data Plane: Esecuzione, Scheduling e Rete

[**Executor.java**]

(file:///home/chiara/DS_Project_JAVA/DS_Project/src/main/java/Executor.java)

- **Ruolo:** Implementa l'oggetto remoto RMI. Gestisce la sottomissione dei client, lo scheduling guidato dal consenso e l'esecuzione multithread dei job.
- **Strutture Dati e Componenti:**
 - `threadPool` : pool fisso di 4 thread per eseguire `job.execute()` in locale.
 - `activeJobsCount` : contatore atomico dei job in elaborazione.
 - `schedulerService` : timer periodico (1 sec) che invoca `schedulePendingJobs()` .
- **Flussi Principali:**
 - **Sottomissione (`submitJob`):**

- Se Follower: effettua il **Transparent Forwarding** al Leader via RMI.
- Se Leader: controlla la deduplicazione client, propone `CREATE_JOB` nel log Raft, scatena la replica e attende il Quorum con `waitForCommit(entryIndex, 5000)` prima di rispondere al client.
- **Scheduler del Leader (`schedulePendingJobs`):**
 - *Failover*: se un job è `ASSIGNED` o `RUNNING` su un nodo non più vivo (`!clusterManager.isNodeAlive(...)`), propone `REQUEUE_JOB`.
 - *Bilanciamento (Min-Load)*: calcola il carico effettivo di tutti i nodi vivi direttamente dalla State Machine, individua il nodo con minor carico e propone `ASSIGN_JOB(jobId, [chosenNode, attempt])`. A commit avvenuto, contatta il worker via RMI (`remoteWorker.executeJob(...)`).
- **Esecuzione Worker (`executeJob`):**
 - Sottomette il calcolo al proprio `threadPool`.
 - Esegue il task e notifica il Leader tramite `notifyJobCompletion(...)`.
- **Completamento (`notifyJobCompletion`):**
 - Se ricevuta dal Leader: verifica che l'attempt corrisponda e propone `COMPLETE_JOB` (o `FAIL_JOB`) nel log Raft.
- **Consultazione Risultato (`getJobResult`):**
 - Interroga la `JobStateMachine` locale in tempo $O(1)$. Poiché Raft replica tutti i completamenti, qualsiasi nodo può rispondere istantaneamente al Client.

[`UDPMulticastSender.java`]

(file:///home/chiara/DS_Project_JAVA/DS_Project/src/main/java/UDPMulticastSender.java)

- **Ruolo:** Thread daemon che trasmette periodicamente la presenza del nodo via UDP Multicast.
- **Funzionamento:**
Ogni 3 secondi invia un datagramma sul gruppo `230.0.0.0:4446` con payload: `nodeId, ipAddress, rmiPort, activeJobs, currentTerm, isLeader`.

[`UDPMulticastReceiver.java`]

(file:///home/chiara/DS_Project_JAVA/DS_Project/src/main/java/UDPMulticastReceiver.java)

- **Ruolo:** Thread daemon in ascolto continuo dei battiti cardiaci UDP degli altri nodi.
- **Funzionamento:**
Effettua la `joinGroup(230.0.0.0)` e riceve i pacchetti. Per ogni pacchetto proveniente da un nodo diverso da sé, aggiorna la tabella dei nodi attivi invocando `clusterManager.isAlive(nodeInfo)`.

[`Main.java`](file:///home/chiara/DS_Project_JAVA/DS_Project/src/main/java/Main.java)

- **Ruolo:** Entry point per l'avvio di un nodo server del cluster.
- **Funzionamento:**
 1. Legge gli argomenti da linea di comando (`nodeId, rmiPort, expectedClusterSize`).
 2. Imposta l'indirizzo IP pubblico RMI (`java.rmi.server.hostname`).
 3. Istanza `ClusterManager` ed `Executor`.
 4. Crea il registro locale RMI sulla porta designata e pubblica il binding `"Executor"`.
 5. Avvia i thread `UDPMulticastReceiver` e `UDPMulticastSender`.
 6. Avvia un thread di diagnostica che ogni 5 secondi stampa a console Termine, Leader, LastLogIndex e stato locale.

4. I Client e gli Strumenti di Supporto

[`Client.java`](file:///home/chiara/DS_Project_JAVA/DS_Project/src/main/java/Client.java)

- **Ruolo:** Client applicativo interattivo per la sottomissione di un singolo job con tolleranza ai crash.

- **Funzionamento:**

- `discoverLiveNode()` : si mette in ascolto sul multicast UDP e rileva il primo nodo attivo.
- `safeSubmitJob()` : sottomissione resiliente. Se il nodo a cui invia il job crasha (`ConnectException`), intercetta l'eccezione, scopre un nuovo nodo e ripete la chiamata senza interrompersi.
- `safeGetJobResult()` : effettua il polling del risultato con lo stesso meccanismo di failover.

[`ClientTest.java`]

(file:///home/chiera/DS_Progett_JAVA/DS_Project/src/main/java/ClientTest.java)

- **Ruolo:** Suite di stress test distribuito.

- **Funzionamento:**

- Sottomette 10 job in sequenza rapida, ciascuno dotato di un progressivo `requestId` (da 1 a 10) e dello stesso `clientId`.
- Utilizza `safeSubmitJob` e `safeGetJobResult` : se durante il test uno o più nodi vengono terminati con `Ctrl+C`, il client scopre automaticamente i superstiti e porta a termine con successo tutti i 10 job, stampando statistiche di latenza e tempi di esecuzione.

[`LogViewer.java`]

(file:///home/chiera/DS_Progett_JAVA/DS_Project/src/main/java/LogViewer.java)

- **Ruolo:** Strumento di diagnostica a riga di comando per ispezionare i log binari `<nodeId>_raft_log.dat`.

- **Funzionamento:**

- Legge e deserializza il file `.dat` specificato.
- Formatta a video una tabella dettagliata con `INDEX`, `TERM`, `COMMAND TYPE`, `JOB ID` e `PAYLOAD` (destinatario dell'assegnazione, tentativo, esito del calcolo).
- Viene invocato rapidamente tramite lo script `./view-log.sh <nodeId>`.

Riepilogo delle Interazioni tra Classi

